Host Runtime

0.8.0

Generated by Doxygen 1.9.1

1 Module Index 1
1.1 Modules . . . 1

2 Namespace Index 3
2.1 Namespace List . . . 3

3 Hierarchical Index 5
3.1 Class Hierarchy . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 Module Documentation 9
5.1 Runtime Profile Event API . . . 9
5.2 Runtime Profiler API . . . 9
5.3 Runtime API . . . 9
5.4 Runtime Types . . . 9

6 Namespace Documentation 11
6.1 rt::profiling Namespace Reference . . . 11
6.1.1 Detailed Description . . . 12

7 Class Documentation 13
7.1 dev::DeviceLayerFake Class Reference . . . 13
7.2 rt::DeviceProperties Struct Reference . . . 14
7.2.1 Detailed Description . . . 15
7.3 rt::DmaInfo Struct Reference . . . 15
7.4 rt::Exception Class Reference . . . 15
7.4.1 Detailed Description . . . 15
7.5 rt::IProfiler Class Reference . . . 16
7.5.1 Detailed Description . . . 16
7.5.2 Member Function Documentation . . . 16
7.5.2.1 start() . . . 16
7.6 rt::profiling::IProfilerRecorder Class Reference . . . 17
7.6.1 Detailed Description . . . 17
7.7 rt::IRuntime Class Reference . . . 17
7.7.1 Detailed Description . . . 19
7.7.2 Member Function Documentation . . . 19
7.7.2.1 abortCommand() . . . 20
7.7.2.2 abortStream() . . . 21
7.7.2.3 create() [1/2] . . . 21
7.7.2.4 create() [2/2] . . . 22
7.7.2.5 createStream() . . . 22
7.7.2.6 destroyStream() . . . 22

7.7.2.7 freeDevice() . . . 23
7.7.2.8 getDeviceProperties() . . . 23
7.7.2.9 getDevices() . . . 23
7.7.2.10 getDmaInfo() . . . 23
7.7.2.11 getProfiler() . . . 24
7.7.2.12 kernelLaunch() . . . 24
7.7.2.13 loadCode() . . . 25
7.7.2.14 mallocDevice() . . . 26
7.7.2.15 memcpyDeviceToHost() [1/2] . . . 26
7.7.2.16 memcpyDeviceToHost() [2/2] . . . 27
7.7.2.17 memcpyHostToDevice() [1/2] . . . 27
7.7.2.18 memcpyHostToDevice() [2/2] . . . 28
7.7.2.19 retrieveStreamErrors() . . . 29
7.7.2.20 setOnKernelAbortedErrorCallback() . . . 29
7.7.2.21 setOnStreamErrorsCallback() . . . 29
7.7.2.22 unloadCode() . . . 30
7.7.2.23 waitForEvent() . . . 30
7.7.2.24 waitForStream() . . . 30
7.8 rt::LoadCodeResult Struct Reference . . . 31
7.8.1 Detailed Description . . . 31
7.9 rt::MemcpyList Struct Reference . . . 31
7.9.1 Detailed Description . . . 32
7.10 rt::MemcpyList::Op Struct Reference . . . 32
7.11 rt::Options Struct Reference . . . 32
7.11.1 Detailed Description . . . 32
7.11.2 Member Data Documentation . . . 32
7.11.2.1 checkDeviceApiVersion_ . . . 33
7.12 rt::profiling::ProfileEvent Class Reference . . . 33
7.12.1 Detailed Description . . . 34
7.13 rt::StreamError Struct Reference . . . 34
7.13.1 Detailed Description . . . 35
7.14 rt::UserTrace Struct Reference . . . 35
7.14.1 Member Data Documentation . . . 35
7.14.1.1 buffer_ . . . 35
7.14.1.2 buffer_size_ . . . 35
7.14.1.3 eventMask_ . . . 35
7.14.1.4 filterMask_ . . . 36
7.14.1.5 shireMask_ . . . 36
7.14.1.6 threadMask_ . . . 36
7.14.1.7 threshold_ . . . 36

Index **37**

Chapter 1

Module Index

1.1 Modules

Here is a list of all modules:

Runtime Profile Event API 9
Runtime Profiler API 9
Runtime API 9
Runtime Types 9

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

rt::profiling
Forward declaration of IProfilerRecorder 11

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

rt::DeviceProperties 14
rt::DmaInfo 15
IDeviceLayer
  dev::DeviceLayerFake 13
rt::IProfiler 16
  rt::profiling::IProfilerRecorder 17
rt::IRuntime 17
rt::LoadCodeResult 31
rt::MemcpyList 31
rt::MemcpyList::Op 32
rt::Options 32
rt::profiling::ProfileEvent 33
dbg::StackException
  rt::Exception 15
rt::StreamError 34
rt::UserTrace 35

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

dev::DeviceLayerFake 13

rt::DeviceProperties
This struct encodes device properties 14

rt::DmaInfo 15

rt::Exception
The error handling in the runtime is trough exceptions. This is the only exception kind that the runtime can throw 15

rt::IProfiler
IProfiler interface declaration, all IProfiler interactions should be made using this interface . . . 16

rt::profiling::IProfilerRecorder
IProfilerRecorder interface declaration, its a simple added method to IProfiler interface for recording 17

rt::IRuntime
Facade Runtime interface declaration, all runtime interactions should be made using this interface. There is a static method create to make runtime instances (factory method) 17

rt::LoadCodeResult
This struct will contain the results of calling a loadCode function 31

rt::MemcpyList
This struct will hold a number of memcpy operations 31

rt::MemcpyList::Op 32

rt::Options
This struct will hold parametrization options for Runtime instantiation 32

rt::profiling::ProfileEvent
ProfileEvent exposes serialization internals for runtime users 33

rt::StreamError
This struct contains the errorCode given by de device when some command fail and the associated ErrorContext (if any) 34

rt::UserTrace 35

Chapter 5

Module Documentation

5.1 Runtime Profile Event API

The runtime profiler API allows to gather profiling data from runtime execution. This API expects to receive an output stream where all profiling events will be dumped, the user can explicitely indicate when to start pofiling and when to stop profiling.

5.2 Runtime Profiler API

The runtime profiler API allows to gather profiling data from runtime execution. This API expects to receive an output stream where all profiling events will be dumped, the user can explicitely indicate when to start pofiling and when to stop profiling.

5.3 Runtime API

The runtime API provides different services to the et-soc devices, allowing the user to manage device memory (see mallocDeviceId and freeDeviceId), load executable elfs into the device (see loadCode and unloadCode) and execute workloads in an asynchronous fashion. These workloads typically consist on copying memory from the host to the device (see memcpyHostToDeviceId), launching a kernel (see kernelLaunch) and getting the results back from the device to the host (see memcpyDeviceIdToHost) Different workloads can be run asynchronously and potentially in parallel, using streams. Every operation is asynchronous, there are also primitives to allow synchronization between the host and the device (see waitForEventId and streamFlush)

5.4 Runtime Types

These are custom types declared by Runtime

Chapter 6

Namespace Documentation

6.1 rt::profiling Namespace Reference

Forward declaration of IProfilerRecorder.

Classes

- class ProfileEvent

  ProfileEvent exposes serialization internals for runtime users.
- class IProfilerRecorder

  IProfilerRecorder interface declaration, its a simple added method to IProfiler interface for recording.

Enumerations

- enum class **Version** : uint16_t
- enum class **Type** { **Start** , **End** , **Complete** , **Instant** }
- enum class **Class** {
  GetDevices , **LoadCode** , **UnloadCode** , **MallocDevice** ,
  FreeDevice , **CreateStream** , **DestroyStream** , **KernelLaunch** ,
  MemcpyHostToDevice , **MemcpyDeviceToHost** , **WaitForEvent** , **WaitForStream** ,
  DeviceCommand , **Pmc** , **CommandSent** , **ResponseReceived** ,
  DispatchEvent , **GetDeviceProperties** , **StartProfiling** , **EndProfiling** ,
  COUNT }
- enum class **ResponseType** { **DMARead** , **DMAWrite** , **Kernel** , **COUNT** }

Functions

- Class **class_from_string** (const std::string &str)
- Type **type_from_string** (const std::string &str)
- ResponseType **response_type_from_string** (const std::string &str)
- std::string **getString** (rt::profiling::Class cls)
- std::string **getString** (rt::profiling::Type type)
- std::string **getString** (rt::profiling::ResponseType rspType)
- template<typename Archive >
  void **serialize** (Archive &ar, rt::EventId &id)

- template<typename Archive >
  void **serialize** (Archive &ar, rt::StreamId &id)
- template<typename Archive >
  void **serialize** (Archive &ar, rt::DeviceId &id)
- template<typename Archive >
  void **serialize** (Archive &ar, rt::KernelId &id)
- template<class Archive >
  void **load** (Archive &ar, ProfileEvent &evt)
- template<class Archive >
  void **save** (Archive &ar, const ProfileEvent &evt)

Variables

- constexpr auto **kCurrentVersion** = Version{1}

6.1.1 Detailed Description

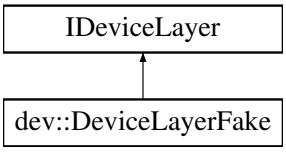
Forward declaration of IProfilerRecorder.

Chapter 7

Class Documentation

7.1 dev::DeviceLayerFake Class Reference

Inheritance diagram for dev::DeviceLayerFake:



Public Member Functions

- **DeviceLayerFake** (int numDevices=1)
- bool **sendCommandMasterMinion** (int device, int, std::byte ∗command, size_t, dev::CmdFlagMM) override
- void **setSqThresholdMasterMinion** (int, int, uint32_t) override
- void **waitForEpollEventsMasterMinion** (int device, uint64_t &sq_bitmap, bool &cq_available, std::chrono↩ ::milliseconds timeout) override
- bool **receiveResponseMasterMinion** (int device, std::vector< std::byte > &response) override
- bool **sendCommandServiceProcessor** (int device, std::byte ∗command, size_t, CmdFlagSP) override
- void **setSqThresholdServiceProcessor** (int, uint32_t) override
- void **waitForEpollEventsServiceProcessor** (int device, bool &sq_available, bool &cq_available, std↩ ::chrono::milliseconds timeout) override
- bool **receiveResponseServiceProcessor** (int device, std::vector< std::byte > &response) override
- int **getDevicesCount** () const override
- DeviceState **getDeviceStateMasterMinion** (int) const override
- DeviceState **getDeviceStateServiceProcessor** (int) const override
- int **getSubmissionQueuesCount** (int) const override
- size_t **getSubmissionQueueSizeMasterMinion** (int) const override
- size_t **getSubmissionQueueSizeServiceProcessor** (int) const override
- int **getDmaAlignment** () const override
- DmaInfo **getDmaInfo** (int) const override
- uint64_t **getDramSize** () const override
- uint64_t **getDramBaseAddress** () const override
- void ∗ **allocDmaBuffer** (int, size_t sizeInBytes, bool) override
- void **freeDmaBuffer** (void ∗dmaBuffer) override
- size_t **getTraceBufferSizeMasterMinion** (int, TraceBufferType traceType) override

- bool **getTraceBufferServiceProcessor** (int, TraceBufferType, std::vector< std::byte > &) override
- DeviceConfig **getDeviceConfig** (int) override
- int **getActiveShiresNum** (int device) override
- uint32_t **getFrequencyMHz** (int device) override
- int **updateFirmwareImage** (int, std::vector< unsigned char > &) override
- size_t **getFreeCmaMemory** () const override
- std::string **getDeviceAttribute** (int, std::string) const override
- void **clearDeviceAttributes** (int, std::string) const override
- void **hintInactivity** (int) override

The documentation for this class was generated from the following file:

- DeviceLayerFake.h

7.2 rt::DeviceProperties Struct Reference

This struct encodes device properties.

`#include <Types.h>`

Public Types

- enum class **FormFactor** { **PCIE** , **M2** }
- enum class **ArchRevision** { **ETSOC1** , **PANTERO** , **GEPARDO** , **UNKNOWN** = -1 }

Public Attributes

- uint32_t frequency_

  minion boot frequency in Mhz
- uint32_t availableShires_

  device shire count
- uint32_t memoryBandwidth_

  device memory bandwidth in MBytes/second
- uint64_t memorySize_

  device memory size in bytes
- uint16_t l3Size_

  device L3 size in bytes
- uint16_t l2shireSize_

  device L2 shire size in bytes
- uint16_t l2scratchpadSize_

  device L2 scratchpad size in bytes
- uint16_t cacheLineSize_

  device caceh line size in bytes
- uint16_t l2CacheBanks_

  number of banks in the L2
- uint32_t computeMinionShireMask_

  mask which indicates what are the compute minion shires
- uint16_t spareComputeMinionoShireId_

  spare compute minion Shire ID
- ArchRevision deviceArch_

  device architecture revision
- FormFactor formFactor_

  device form factor
- uint8_t tdp_

  TDP in Watts.

7.2.1 Detailed Description

This struct encodes device properties.

The documentation for this struct was generated from the following file:

- Types.h

7.3 rt::DmaInfo Struct Reference

Public Attributes

- uint64_t maxElementSize_

  maximum amount of memory that can be transfer per each DMA command entry
- uint64_t maxElementCount_

  max number of DMA entries per DMA command

The documentation for this struct was generated from the following file:

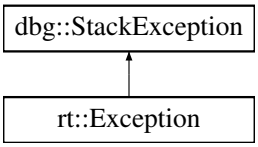
- Types.h

7.4 rt::Exception Class Reference

The error handling in the runtime is trough exceptions. This is the only exception kind that the runtime can throw.

```
#include <Types.h>
```

Inheritance diagram for rt::Exception:



7.4.1 Detailed Description

The error handling in the runtime is trough exceptions. This is the only exception kind that the runtime can throw.

The documentation for this class was generated from the following file:

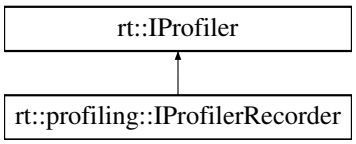
- Types.h

7.5 rt::IProfiler Class Reference

IProfiler interface declaration, all IProfiler interactions should be made using this interface.

```
#include <IProfiler.h>
```

Inheritance diagram for rt::IProfiler:



Public Types

- enum class OutputType { **Json** , **Binary** }

  Profiler user can choose what profiling output generate, Json format or Binary format.

Public Member Functions

- virtual ~IProfiler ()=default

  Virtual Destructor to enable polymorphic release of IProfiler instances.
- virtual void start (std::ostream &outputStream, OutputType outputType)=0

  Indicates the start of the profiler. The given stream will be accessed freely from the runtime, don't make concurrent writes to this stream or data races could happen.
- virtual void stop ()=0

  This function will stop the profiler from gathering data.

7.5.1 Detailed Description

IProfiler interface declaration, all IProfiler interactions should be made using this interface.

7.5.2 Member Function Documentation

7.5.2.1 start()

```
virtual void rt::IProfiler::start (
            std::ostream & outputStream,
            OutputType outputType )  [pure virtual]
```

Indicates the start of the profiler. The given stream will be accessed freely from the runtime, don't make concurrent writes to this stream or data races could happen.

Parameters

in	*outputStream*	this is the stream where the profiler will dump the data. Make sure its opened in binary mode.
in	*outputType*	to choose between different formatting options for the outputstream OutputType

The documentation for this class was generated from the following file:

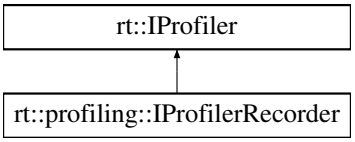
- IProfiler.h

7.6 rt::profiling::IProfilerRecorder Class Reference

IProfilerRecorder interface declaration, its a simple added method to IProfiler interface for recording.

```
#include <IProfiler.h>
```

Inheritance diagram for rt::profiling::IProfilerRecorder:



Public Member Functions

- virtual void **record** (const ProfileEvent &event)=0

Additional Inherited Members

7.6.1 Detailed Description

IProfilerRecorder interface declaration, its a simple added method to IProfiler interface for recording.

The documentation for this class was generated from the following file:

- IProfiler.h

7.7 rt::IRuntime Class Reference

Facade Runtime interface declaration, all runtime interactions should be made using this interface. There is a static method create to make runtime instances (factory method)

```
#include <IRuntime.h>
```

Public Member Functions

- std::vector< DeviceId > getDevices ()

  Returns all devices.
- DeviceProperties getDeviceProperties (DeviceId device) const

  Returns the properties of a given device.
- std::byte * mallocDevice (DeviceId device, size_t size, uint32_t alignment=kCacheLineSize)

  Allocates memory in the device, returns a device memory pointer. One can't use this pointer directly from the host, this pointer is intended to be used for memory operations between the host and the device.
- void freeDevice (DeviceId device, std::byte *buffer)

  Deallocates previously allocated memory on the given device.
- StreamId createStream (DeviceId device)

  Creates a new stream and associates it to the given device. A stream is an abstraction of a "pipeline" where you can push operations (mem copies or kernel launches) and enforce the dependencies between these operations.
- void destroyStream (StreamId stream)

  Destroys a previously created stream.
- LoadCodeResult loadCode (StreamId stream, const std::byte *elf, size_t elf_size)

  Loads an elf into the device. The caller will provide a byte code containing the elf representation and its size. Host memory.
- void unloadCode (KernelId kernel)

  Unloads a previously loaded elf code, identified by the kernel handler.
- EventId kernelLaunch (StreamId stream, KernelId kernel, const std::byte *kernel_args, size_t kernel_args↩_size, uint64_t shire_mask, bool barrier=true, bool flushL3=false, std::optional< UserTrace > userTrace↩Config=std::nullopt)

  Queues a execution work into a stream. The work is identified by the kernel handler, which has been previously loaded into the device which is associated to the stream. The parameters of the kernel are given by kernel_args and kernel_args_size; these parameters will be copied from the host memory to the device memory by the runtime. The firmware will load these parameters into RA register, the kernel code should cast this register to the expected types. The kernel execution is always asynchronous.
- EventId memcpyHostToDevice (StreamId stream, const std::byte *h_src, std::byte *d_dst, size_t size, bool barrier=false, const CmaCopyFunction &cmaCopyFunction=defaultCmaCopyFunction)

  Queues a memcpy operation from host memory to device memory. The device memory must be previously allocated by a mallocDevice.
- EventId memcpyDeviceToHost (StreamId stream, const std::byte *d_src, std::byte *h_dst, size_t size, bool barrier=true, const CmaCopyFunction &cmaCopyFunction=defaultCmaCopyFunction)

  Queues a memcpy operation from device memory to host memory. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap)
- EventId memcpyHostToDevice (StreamId stream, MemcpyList memcpyList, bool barrier=false, const Cma↩CopyFunction &cmaCopyFunction=defaultCmaCopyFunction)

  Queues many memcpy operations from host memory to device memory. These operations are defined in struct MemcpyList. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap).
- EventId memcpyDeviceToHost (StreamId stream, MemcpyList memcpyList, bool barrier=true, const Cma↩CopyFunction &cmaCopyFunction=defaultCmaCopyFunction)

  Queues many memcpy operations from device memory to host memory. These operations are defined in struct MemcpyList. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap).
- bool waitForEvent (EventId event, std::chrono::seconds timeout=std::chrono::hours(24))

  This will block the caller thread until the given event is dispatched or the timeout is reached. This primitive allows to synchronize with the device execution.
- bool waitForStream (StreamId stream, std::chrono::seconds timeout=std::chrono::hours(24))

  This will block the caller thread until all commands issued to the given stream finish or if the timeout is reached. This primitive allows to synchronize with the device execution.
- std::vector< StreamError > retrieveStreamErrors (StreamId stream)

This will return a list of errors and their execution context (if any)

- void setOnKernelAbortedErrorCallback (const KernelAbortedCallback &callback)

  This callback (when set) will be automatically called when a kernel abort happens. This is implemented as a workaround (SW-13045).

- void setOnStreamErrorsCallback (StreamErrorCallback callback)

  This callback (when set) will be automatically called when a new StreamError occurs, making the polling through retrieveStreamErrors unnecessary.

- void setProfiler (std::unique_ptr< profiling::IProfilerRecorder > profiler)

  Sets a profiler to be used by runtime.

- profiling::IProfilerRecorder * getProfiler () const

  Returns a pointer to the profiler interface; don't delete/free this pointer since this is owned by the runtime itself.

- EventId abortCommand (EventId commandId, std::chrono::milliseconds timeout=std::chrono::milliseconds(5000))

  Instructs the device to abort given command. NOTE: as per current firmware implementation, aborting a command will have undesirable side effects on the rest of the submitted commands to the same virtual queue. So, after aborting a command it could potentially affect the rest of the executions BUG: individual memcpyHostToDevice and memcpy↩ DeviceToHost commands can not be aborted individually right now. The could be aborted through abortStream.

- EventId abortStream (StreamId streamId)

  Instructs the device to abort all outstanding commands on a given stream. This will affect to all commands previously issued to given stream, later commands will execute normally. NOTE: as per current firmware implementation, aborting a command will have undesirable side effects on the rest of the submitted commands to the same virtual queue. So, after aborting a command it could potentially affect the rest of the executions.

- DmaInfo getDmaInfo (DeviceId deviceId) const

  Returns the DmaInfo of given device (see DmaInfo). This DmaInfo contains max size for each operation in a memcpy list command and max number of operations allowed in a single memcpy list command.

- virtual ~IRuntime ()=default

  Virtual Destructor to enable polymorphic release of the runtime instances.

- IRuntime ()

  Constructor to initialize with a dummy profiler.

Static Public Member Functions

- static RuntimePtr create (dev::IDeviceLayer *deviceLayer, Options options=getDefaultOptions())

  Factory method to instantiate a standalone IRuntime implementation.

- static RuntimePtr create (const std::string &socketPath)

  Factory method to instantiate a client IRuntime implementation.

7.7.1 Detailed Description

Facade Runtime interface declaration, all runtime interactions should be made using this interface. There is a static method create to make runtime instances (factory method)

7.7.2 Member Function Documentation

7.7.2.1 abortCommand()

```
EventId rt::IRuntime::abortCommand (
            EventId commandId,
            std::chrono::milliseconds timeout = std::chrono::milliseconds(5000) )
```

Instructs the device to abort given command. NOTE: as per current firmware implementation, aborting a command will have undesirable side effects on the rest of the submitted commands to the same virtual queue. So, after aborting a command it could potentially affect the rest of the executions BUG: individual memcpyHostToDevice and memcpyDeviceToHost commands can not be aborted individually right now. The could be aborted through abortStream.

Parameters

in	*command↩ Id*	indicates the command's eventId to abort
in	*timeout*	indicates the maximum time trying to QUEUE an abort command into the device. If the timeout is reached and the runtime was not able to push the command into the device, an Exception will be thrown

Returns

EventId is a handler of the abort command itself which can be waited for (waitForEventId) to synchronize.

7.7.2.2 abortStream()

```
EventId rt::IRuntime::abortStream (
            StreamId streamId )
```

Instructs the device to abort all outstanding commands on a given stream. This will affect to all commands previously issued to given stream, later commands will execute normally. NOTE: as per current firmware implementation, aborting a command will have undesirable side effects on the rest of the submitted commands to the same virtual queue. So, after aborting a command it could potentially affect the rest of the executions.

Parameters

in	*stream↩ Id*	indicates the stream to abort commands

Returns

EventId is a handler of the abortStream which can be waited for (waitForEventId) to synchronize.

7.7.2.3 create() [1/2]

```
static RuntimePtr rt::IRuntime::create (
            const std::string & socketPath )  [static]
```

Factory method to instantiate a client IRuntime implementation.

Parameters

in	*socketPath*	indicates which socket the Client will connect to

Returns

RuntimePtr an IRuntime instance. See dev::IDeviceLayer

7.7.2.4 create() [2/2]

```
static RuntimePtr rt::IRuntime::create (
            dev::IDeviceLayer * deviceLayer,
            Options options = getDefaultOptions() )  [static]
```

Factory method to instantiate a standalone IRuntime implementation.

Parameters

in	*Options*	can set some runtime parameters. See Options

Returns

RuntimePtr an IRuntime instance. See dev::IDeviceLayer

7.7.2.5 createStream()

```
StreamId rt::IRuntime::createStream (
            DeviceId device )
```

Creates a new stream and associates it to the given device. A stream is an abstraction of a "pipeline" where you can push operations (mem copies or kernel launches) and enforce the dependencies between these operations.

Parameters

in	*device*	handler indicating in which device to associate the stream

Returns

a stream handler

7.7.2.6 destroyStream()

```
void rt::IRuntime::destroyStream (
            StreamId stream )
```

Destroys a previously created stream.

Parameters

in	*stream*	handler to the stream to be destroyed

7.7.2.7 freeDevice()

```
void rt::IRuntime::freeDevice (
            DeviceId device,
            std::byte * buffer )
```

Deallocates previously allocated memory on the given device.

Parameters

in	*device*	handler indicating in which device to deallocate the memory
in	*buffer*	device memory pointer previously allocated with mallocDevice to be deallocated

7.7.2.8 getDeviceProperties()

```
DeviceProperties rt::IRuntime::getDeviceProperties (
            DeviceId device ) const
```

Returns the properties of a given device.

Parameters

in	*device*	handler indicating the device

Returns

the properties for the requested device

7.7.2.9 getDevices()

```
std::vector<DeviceId> rt::IRuntime::getDevices ( )
```

Returns all devices.

Returns

a vector containing all device handlers

7.7.2.10 getDmaInfo()

```
DmaInfo rt::IRuntime::getDmaInfo (
            DeviceId deviceId ) const
```

Returns the DmaInfo of given device (see DmaInfo). This DmaInfo contains max size for each operation in a memcpy list command and max number of operations allowed in a single memcpy list command.

Parameters

in	*device↩ Id*	indicates the device to get the DMA configuration from.

Returns

DmaInfo contains max number of operations allowed in a single memcpy list command and max size for each operation.

7.7.2.11 getProfiler()

```
profiling::IProfilerRecorder* rt::IRuntime::getProfiler ( ) const  [inline]
```

Returns a pointer to the profiler interface; don't delete/free this pointer since this is owned by the runtime itself.

Returns

IProfilerRecorder an interface to the profiler. See IProfilerRecorder

7.7.2.12 kernelLaunch()

```
EventId rt::IRuntime::kernelLaunch (
            StreamId stream,
            KernelId kernel,
            const std::byte * kernel_args,
            size_t kernel_args_size,
            uint64_t shire_mask,
            bool barrier = true,
            bool flushL3 = false,
            std::optional< UserTrace > userTraceConfig = std::nullopt )
```

Queues a execution work into a stream. The work is identified by the kernel handler, which has been previously loaded into the device which is associated to the stream. The parameters of the kernel are given by kernel_args and kernel_args_size; these parameters will be copied from the host memory to the device memory by the runtime. The firmware will load these parameters into RA register, the kernel code should cast this register to the expected types. The kernel execution is always asynchronous.

Parameters

in	*stream*	handler indicating in which stream the kernel will be executed. The kernel code have to be registered into the device associated to the stream previously.
in	*kernel*	handler which indicate what code to execute in the device.
in	*kernel_args*	buffer containing all the parameters to be copied to the device memory prior to the code execution
in	*kernel_args_size*	size of the kernel_args buffer
in	*shire_mask*	indicates in what shires the kernel will be executed

Parameters

in	*barrier*	this parameter indicates if the kernel execution should be postponed till all previous works issued into this stream finish (a barrier). Usually the kernel launch must be postponed till some previous memory operations end, hence the default value is true.
in	*flushL3*	this parameter indicates if the L3 should be flushed before the kernel execution starts.
in	*userTraceBuffer*	this parameter can be null or point to a device buffer (previously allocated with mallocDevice). If the pointer is not null, then the firmware will utilize this buffer to fill-up user trace data. This buffer must be of size 4KB $*$ num_harts (4KB$*$2080). We will provide later on an API to allocate the buffer with the size required.

Returns

EventId is a handler of an event which can be waited for (waitForEventId) to syncrhonize when the kernel ends the execution.

7.7.2.13 loadCode()

```
LoadCodeResult rt::IRuntime::loadCode (
            StreamId stream,
            const std::byte * elf,
            size_t elf_size )
```

Loads an elf into the device. The caller will provide a byte code containing the elf representation and its size. Host memory.

Parameters

in	*stream*	handler indicating the stream used for the kernel loading.
in	*elf*	a pointer to host memory containing the elf bytes
in	*elf_size*	the elf size

Returns

a LoadCodeResult with the EventId to sync with (if needed), the kernelId to utilize in later kernelLaunch and the kernel load address.

NOTE: remember to not deallocate the elf memory

Parameters

elf	until the EventId from LoadCodeResult is completed

7.7.2.14 mallocDevice()

```
std::byte* rt::IRuntime::mallocDevice (
            DeviceId device,
            size_t size,
            uint32_t alignment = kCacheLineSize )
```

Allocates memory in the device, returns a device memory pointer. One can't use this pointer directly from the host, this pointer is intended to be used for memory operations between the host and the device.

Parameters

in	*device*	handler indicating in which device to allocate the memory
in	*size*	indicates the memory allocation size in bytes
in	*alignment*	indicates the required alignment for memory allocation, defaults to device cache line size

Returns

a device memory pointer

7.7.2.15 memcpyDeviceToHost() [1/2]

```
EventId rt::IRuntime::memcpyDeviceToHost (
            StreamId stream,
            const std::byte * d_src,
            std::byte * h_dst,
            size_t size,
            bool barrier = true,
            const CmaCopyFunction & cmaCopyFunction = defaultCmaCopyFunction )
```

Queues a memcpy operation from device memory to host memory. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap)

Parameters

in	*stream*	handler indicating in which stream to queue the memcpy operation
in	*d_src*	device memory buffer to copy from
in	*h_dst*	host memory buffer to copy to
in	*size*	indicates the size of the memcpy
in	*barrier*	this parameter indicates if the memcpy operation should be postponed till all previous works issued into this stream finish (a barrier). Usually the memcpies from device to host must wait till a previous kernel execution finishes, hence the default value is true All memcpy operations are always asynchronous.
in	*cmaCopyFunction*	this parameter allows to customize the function used to copy from user-space virtual memory to the CMA buffer. Intended for internal usage, regular API user shouldn't modify the default value.

Returns

EventId is a handler of an event which can be waited for (waitForEventId) to synchronize when the memcpy ends.

NOTE: the host memory pointer must be kept alive until the operation has completely ended in device.

7.7.2.16 memcpyDeviceToHost() [2/2]

```
EventId rt::IRuntime::memcpyDeviceToHost (
            StreamId stream,
            MemcpyList memcpyList,
            bool barrier = true,
            const CmaCopyFunction & cmaCopyFunction = defaultCmaCopyFunction )
```

Queues many memcpy operations from device memory to host memory. These operations are defined in struct MemcpyList. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap).

Parameters

in	*stream*	handler indicating in which stream to queue the memcpy operation
in	*memcpyList*	contains all the operations required. See MemcpyList
in	*barrier*	this parameter indicates if the memcpy operation should be postponed till all previous works issued into this stream finish (a barrier). Usually the memcpies from device to host must wait till a previous kernel execution finishes, hence the default value is true All memcpy operations are always asynchronous.
in	*cmaCopyFunction*	this parameter allows to customize the function used to copy from user-space virtual memory to the CMA buffer. Intended for internal usage, regular API user shouldn't modify the default value.

Returns

EventId is a handler of an event which can be waited for (waitForEventId) to synchronize when the memcpy ends.

7.7.2.17 memcpyHostToDevice() [1/2]

```
EventId rt::IRuntime::memcpyHostToDevice (
            StreamId stream,
            const std::byte * h_src,
            std::byte * d_dst,
            size_t size,
            bool barrier = false,
            const CmaCopyFunction & cmaCopyFunction = defaultCmaCopyFunction )
```

Queues a memcpy operation from host memory to device memory. The device memory must be previously allocated by a mallocDevice.

Parameters

in	*stream*	handler indicating in which stream to queue the memcpy operation
in	*h_src*	host memory buffer to copy from
in	*d_dst*	device memory buffer to copy to
in	*size*	indicates the size of the memcpy
in	*barrier*	this parameter indicates if the memcpy operation should be postponed till all previous works issued into this stream finish (a barrier). Usually the memcpies from host to device can run in parallel, hence the default value is false All memcpy operations are always asynchronous.
in	*cmaCopyFunction*	this parameter allows to customize the function used to copy from user-space virtual memory to the CMA buffer. Intended for internal usage, regular API user shouldn't modify the default value.

Returns

EventId is a handler of an event which can be waited for (waitForEventId) to synchronize when the memcpy ends.

NOTE: the host memory pointer must be kept alive until the operation has completely ended in device.

7.7.2.18 memcpyHostToDevice() [2/2]

```
EventId rt::IRuntime::memcpyHostToDevice (
            StreamId stream,
            MemcpyList memcpyList,
            bool barrier = false,
            const CmaCopyFunction & cmaCopyFunction = defaultCmaCopyFunction )
```

Queues many memcpy operations from host memory to device memory. These operations are defined in struct MemcpyList. The device memory must be a valid region previously allocated by a mallocDevice; the host memory must be a previously allocated memory in the host by the conventional means (for example the heap).

Parameters

in	*stream*	handler indicating in which stream to queue the memcpy operation
in	*memcpyList*	contains all the operations required. See MemcpyList
in	*barrier*	this parameter indicates if the memcpy operation should be postponed till all previous works issued into this stream finish (a barrier). Usually the memcpies from device to host must wait till a previous kernel execution finishes, hence the default value is false. All memcpy operations are always asynchronous.
in	*cmaCopyFunction*	this parameter allows to customize the function used to copy from user-space virtual memory to the CMA buffer. Intended for internal usage, regular API user shouldn't modify the default value.

Returns

EventId is a handler of an event which can be waited for (waitForEventId) to synchronize when the memcpy ends.

7.7.2.19 retrieveStreamErrors()

```
std::vector<StreamError> rt::IRuntime::retrieveStreamErrors (
            StreamId stream )
```

This will return a list of errors and their execution context (if any)

Parameters

in	*stream*	this is the stream to synchronize with.
in	*timeout*	is the number of seconds to wait till aborting the wait.

Returns

StreamStatus contains the state of the associated stream.

7.7.2.20 setOnKernelAbortedErrorCallback()

```
void rt::IRuntime::setOnKernelAbortedErrorCallback (
            const KernelAbortedCallback & callback )
```

This callback (when set) will be automatically called when a kernel abort happens. This is implemented as a workaround (SW-13045).

Parameters

in	*callback*	see KernelAbortedCallback. This is the callback which will be called when a kernel abort happens

7.7.2.21 setOnStreamErrorsCallback()

```
void rt::IRuntime::setOnStreamErrorsCallback (
            StreamErrorCallback callback )
```

This callback (when set) will be automatically called when a new StreamError occurs, making the polling through retrieveStreamErrors unnecessary.

Parameters

in	*callback*	see StreamErrorCallbac. This is the callback which will be called when a StreamError occurs.

7.7.2.22 unloadCode()

```
void rt::IRuntime::unloadCode (
            KernelId kernel )
```

Unloads a previously loaded elf code, identified by the kernel handler.

Parameters

in	*kernel*	a handler to the code that must be unloaded

7.7.2.23 waitForEvent()

```
bool rt::IRuntime::waitForEvent (
            EventId event,
            std::chrono::seconds timeout = std::chrono::hours(24) )
```

This will block the caller thread until the given event is dispatched or the timeout is reached. This primitive allows to synchronize with the device execution.

Parameters

in	*event*	is the event to wait for, result of a memcpy operation or a kernel launch.
in	*timeout*	is the number of seconds to wait till aborting the wait. If timeout is 0 seconds, then it won't block in any case.

Returns

false if the timeout is reached, true otherwise.

7.7.2.24 waitForStream()

```
bool rt::IRuntime::waitForStream (
            StreamId stream,
            std::chrono::seconds timeout = std::chrono::hours(24) )
```

This will block the caller thread until all commands issued to the given stream finish or if the timeout is reached. This primitive allows to synchronize with the device execution.

Parameters

in	*stream*	this is the stream to synchronize with.
in	*timeout*	is the number of seconds to wait till aborting the wait. If timeout is 0 seconds, then it won't block in any case.

Returns

false if the timeout is reached, true otherwise.

The documentation for this class was generated from the following file:

- IRuntime.h

7.8 rt::LoadCodeResult Struct Reference

This struct will contain the results of calling a loadCode function.

```
#include <Types.h>
```

Public Attributes

- EventId **event_**
- KernelId kernel_

  < event to wait for the loadCode is complete
- std::byte * loadAddress_

  < kernelId associated to the loadCode, to be used in later kernelLaunch(es)

7.8.1 Detailed Description

This struct will contain the results of calling a loadCode function.

The documentation for this struct was generated from the following file:

- Types.h

7.9 rt::MemcpyList Struct Reference

This struct will hold a number of memcpy operations.

```
#include <Types.h>
```

Classes

- struct Op

Public Member Functions

- void **addOp** (std::byte *src, std::byte *dst, size_t size)

Public Attributes

- std::vector< Op > **operations_**

7.9.1 Detailed Description

This struct will hold a number of memcpy operations.

The documentation for this struct was generated from the following file:

- Types.h

7.10 rt::MemcpyList::Op Struct Reference

Public Attributes

- std::byte * **src_**
- std::byte * **dst_**
- size_t **size_**

The documentation for this struct was generated from the following file:

- Types.h

7.11 rt::Options Struct Reference

This struct will hold parametrization options for Runtime instantiation.

```
#include <Types.h>
```

Public Attributes

- bool **checkMemcpyDeviceOperations_**
- bool checkDeviceApiVersion_

7.11.1 Detailed Description

This struct will hold parametrization options for Runtime instantiation.

7.11.2 Member Data Documentation

7.11.2.1 checkDeviceApiVersion_

```
bool rt::Options::checkDeviceApiVersion_
```

< if set, the runtime will inspect all memcpy operations and throw an exception if invalid device address/size

The documentation for this struct was generated from the following file:

- Types.h

7.12 rt::profiling::ProfileEvent Class Reference

ProfileEvent exposes serialization internals for runtime users.

```
#include <IProfileEvent.h>
```

Public Types

- using **Id** = uint64_t
- using **Cycles** = uint64_t
- using **Clock** = std::chrono::steady_clock
- using **TimePoint** = std::chrono::time_point< Clock >
- using **Duration** = TimePoint::duration
- using **ExtraValues** = std::variant< uint64_t, EventId, StreamId, DeviceId, KernelId, ResponseType, Duration, DeviceProperties, Version >
- using **ExtraMetadata** = std::unordered_map< std::string, ExtraValues >

Public Member Functions

- **ProfileEvent** (Type type, Class cls)
- **ProfileEvent** (Type type, Class cls, StreamId stream, EventId event, ExtraMetadata extra={})
- Type **getType** () const
- Class **getClass** () const
- TimePoint **getTimeStamp** () const
- std::string **getThreadId** () const
- ExtraMetadata **getExtras** () const
- std::optional< Version > **getVersion** () const
- std::optional< Duration > **getDuration** () const
- std::optional< EventId > **getEvent** () const
- std::optional< EventId > **getParentId** () const
- std::optional< StreamId > **getStream** () const
- std::optional< DeviceId > **getDeviceId** () const
- std::optional< KernelId > **getKernelId** () const
- std::optional< ResponseType > **getResponseType** () const
- std::optional< uint64_t > **getLoadAddress** () const
- std::optional< Cycles > **getDeviceCmdStartTs** () const
- std::optional< Cycles > **getDeviceCmdWaitDur** () const
- std::optional< Cycles > **getDeviceCmdExecDur** () const
- std::optional< DeviceProperties > **getDeviceProperties** () const
- void **setType** (Type t)

- void **setClass** (Class c)
- void **setTimeStamp** (TimePoint t=Clock::now())
- void **setThreadId** (std::thread::id id=std::this_thread::get_id())
- void **setExtras** (ExtraMetadata extras)
- void **setDuration** (Duration d)
- void **setEvent** (EventId event)
- void **setParentId** (EventId parent)
- void **setStream** (StreamId stream)
- void **setDeviceId** (DeviceId deviceId)
- void **setKernelId** (KernelId kernelId)
- void **setResponseType** (ResponseType rspType)
- void **setLoadAddress** (uint64_t loadAddress)
- void **setDeviceCmdStartTs** (uint64_t start_ts)
- void **setDeviceCmdWaitDur** (uint64_t wait_dur)
- void **setDeviceCmdExecDur** (uint64_t exec_dur)
- void **setDeviceProperties** (DeviceProperties props)

Friends

- template<class Archive >
  void **load** (Archive &ar, ProfileEvent &evt)

7.12.1 Detailed Description

ProfileEvent exposes serialization internals for runtime users.

The documentation for this class was generated from the following file:

- IProfileEvent.h

7.13 rt::StreamError Struct Reference

This struct contains the errorCode given by de device when some command fail and the associated ErrorContext (if any)

```
#include <Types.h>
```

Public Member Functions

- **StreamError** (DeviceErrorCode errorCode)
- std::string **getString** () const

Public Attributes

- DeviceErrorCode errorCode_ = DeviceErrorCode::Unknown
  < *returns a string representation of the StreamError*
- std::optional< uint64_t > **cmShireMask_**
- std::optional< std::vector< ErrorContext > > errorContext_
  < *only available in some kernel errors. Contains offending shiremask*

7.13.1 Detailed Description

This struct contains the errorCode given by de device when some command fail and the associated ErrorContext (if any)

The documentation for this struct was generated from the following file:

- Types.h

7.14 rt::UserTrace Struct Reference

Public Attributes

- uint64_t buffer_
- uint32_t buffer_size_
- uint32_t threshold_
- uint64_t shireMask_
- uint64_t threadMask_
- uint32_t eventMask_
- uint32_t filterMask_

7.14.1 Member Data Documentation

7.14.1.1 buffer_

```
uint64_t rt::UserTrace::buffer_
```

Base address for Trace buffer.

7.14.1.2 buffer_size_

```
uint32_t rt::UserTrace::buffer_size_
```

Total size of the Trace buffer.

7.14.1.3 eventMask_

```
uint32_t rt::UserTrace::eventMask_
```

This is a bit mask, each bit corresponds to a specific Event to trace.

7.14.1.4 filterMask_

```
uint32_t rt::UserTrace::filterMask_
```

This is a bit mask representing a list of filters for a given event to trace.

7.14.1.5 shireMask_

```
uint64_t rt::UserTrace::shireMask_
```

Bit Mask of Shire to enable Trace Capture.

7.14.1.6 threadMask_

```
uint64_t rt::UserTrace::threadMask_
```

Bit Mask of Thread within a Shire to enable Trace Capture.

7.14.1.7 threshold_

```
uint32_t rt::UserTrace::threshold_
```

Threshold for free memory in the buffer for each hart.

The documentation for this struct was generated from the following file:

- Types.h

Index

abortCommand
    rt::IRuntime, 19
abortStream
    rt::IRuntime, 21

buffer_
    rt::UserTrace, 35
buffer_size_
    rt::UserTrace, 35

checkDeviceApiVersion_
    rt::Options, 32
create
    rt::IRuntime, 21, 22
createStream
    rt::IRuntime, 22

destroyStream
    rt::IRuntime, 22
dev::DeviceLayerFake, 13

eventMask_
    rt::UserTrace, 35

filterMask_
    rt::UserTrace, 35
freeDevice
    rt::IRuntime, 23

getDeviceProperties
    rt::IRuntime, 23
getDevices
    rt::IRuntime, 23
getDmaInfo
    rt::IRuntime, 23
getProfiler
    rt::IRuntime, 24

kernelLaunch
    rt::IRuntime, 24

loadCode
    rt::IRuntime, 25

mallocDevice
    rt::IRuntime, 25
memcpyDeviceToHost
    rt::IRuntime, 26, 27
memcpyHostToDevice
    rt::IRuntime, 27, 28

retrieveStreamErrors
    rt::IRuntime, 28
rt::DeviceProperties, 14
rt::DmaInfo, 15
rt::Exception, 15
rt::IProfiler, 16
    start, 16
rt::IRuntime, 17
    abortCommand, 19
    abortStream, 21
    create, 21, 22
    createStream, 22
    destroyStream, 22
    freeDevice, 23
    getDeviceProperties, 23
    getDevices, 23
    getDmaInfo, 23
    getProfiler, 24
    kernelLaunch, 24
    loadCode, 25
    mallocDevice, 25
    memcpyDeviceToHost, 26, 27
    memcpyHostToDevice, 27, 28
    retrieveStreamErrors, 28
    setOnKernelAbortedErrorCallback, 29
    setOnStreamErrorsCallback, 29
    unloadCode, 29
    waitForEvent, 30
    waitForStream, 30
rt::LoadCodeResult, 31
rt::MemcpyList, 31
rt::MemcpyList::Op, 32
rt::Options, 32
    checkDeviceApiVersion_, 32
rt::profiling, 11
rt::profiling::IProfilerRecorder, 17
rt::profiling::ProfileEvent, 33
rt::StreamError, 34
rt::UserTrace, 35
    buffer_, 35
    buffer_size_, 35
    eventMask_, 35
    filterMask_, 35
    shireMask_, 36
    threadMask_, 36
    threshold_, 36
Runtime API, 9
Runtime Profile Event API, 9
Runtime Profiler API, 9
Runtime Types, 9

setOnKernelAbortedErrorCallback
rt::IRuntime, 29
setOnStreamErrorsCallback
rt::IRuntime, 29
shireMask_
rt::UserTrace, 36
start
rt::IProfiler, 16

threadMask_
rt::UserTrace, 36
threshold_
rt::UserTrace, 36

unloadCode
rt::IRuntime, 29

waitForEvent
rt::IRuntime, 30
waitForStream
rt::IRuntime, 30